

Merge Intervals — LeetCode 56

Problem

Given an array of intervals `intervals[i] = [start_i, end_i]`, merge all overlapping intervals and return the non-overlapping intervals covering the input.

Example: `[[1,3],[2,6],[8,10],[15,18]] → [[1,6],[8,10],[15,18]]`.

Key observations

- **Sort by start first — that's the whole game.** On unsorted input, a single pass can't work: an interval that doesn't overlap the current one may still overlap a later one (`[[1,2],[10,11],[3,4],[4,10]] → [[1,2],[3,11]]`), and a growing merged interval can swallow intervals you already passed.
- After sorting, overlap is a local check: consecutive intervals `[a,b]`, `[c,d]` overlap iff `c <= b` (note `<=`, not `<`).
- **Touching intervals merge:** `[1,4]` and `[4,5]` `→ [1,5]`. This is the problem's convention, not a judgment call.
- Carry one running merged interval. Extend its end while intervals overlap it; emit it and start fresh on a gap. The merged end is `max(b, d)` — never the min of starts (sorting already handles starts).

Approach

1. Sort by start: $O(n \log n)$.
2. Walk once with a running `[start, end]`: overlap `→ end = max(end, next[1])`; gap `→ emit [start, end]`, reset to the next interval.
3. Emit the final running interval after the loop (the classic forgotten line).

Code

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.Comparator;
import java.util.List;

public class MergeIntervals {
```

```

public static int[][] merge(int[][] intervals) {
    if (intervals == null || intervals.length == 0) {
        return new int[0][];
    }
    Arrays.sort(intervals, Comparator.comparingInt(row -> row[0]));

    List<int[]> merged = new ArrayList<>();
    int start = intervals[0][0];
    int end = intervals[0][1];

    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] <= end) {
            end = Math.max(end, intervals[i][1]); // overlap: extend, don't emit
        } else {
            merged.add(new int[]{start, end}); // gap: emit, start fresh
            start = intervals[i][0];
            end = intervals[i][1];
        }
    }
    merged.add(new int[]{start, end}); // don't forget the last one
    return merged.toArray(new int[merged.size()][]);
}

```

Complexity

- Time: $O(n \log n)$ — the sort dominates; the merge pass is $O(n)$.
- Space: $O(n)$ for the output ($O(1)$ auxiliary besides it).

Edge cases that matter

- Empty / null input → return empty (don't index `intervals[0]` blindly).
- Touching intervals `[1,2], [2,3]` → `[[1,3]]`.
- One interval fully inside another: `[1,10], [2,3]` → `[[1,10]]` (the `max` on the end handles it).
- Single interval, already-sorted input, fully overlapping chain.
- `return` placement: emitting inside the loop instead of after it silently drops everything past the first iteration.

Practice notes — 2026-09-21

What went right: asked the integer clarifying question; stated the merge rule correctly ($\{a, b\}, \{c, d\} \rightarrow \{a, \max(b, d)\}$); found the sort-first insight independently after one counterexample (Day 1 needed a direct hint — this is progress); second attempt had the right structure (sort + running interval + extend-or-emit).

What went wrong: first approach (unsorted single pass with stash-and-merge) broke on chained overlaps and the trace of the counterexample was wrong; first code attempt didn't compile and compared starts instead of extending ends; final code still didn't compile (missing paren, missing imports, `nextIntervals` vs `nextInterval`) and had the `return` inside the `for` loop — it would have emitted after one iteration; no empty-input guard.

Score: 6/10 (10 = exceeds bar). Problem-solving trajectory clearly better than Day 1 — the key insight was self-derived. The gap is now almost entirely code correctness: compiling cleanly and control-flow discipline.

Drills: write-then-trace every loop with the return/emit placement; empty-input guard as reflex; compile in your head (imports, parens, variable names) before calling it done.